bytes used in an array. Because we have an array of the integer data type, `sizeof()` will return twice the number of elements than are actually in the array. When we perform the operation `sizeof(flicker)/2` we will get the value `8`. This can be pretty handy for updating the number of elements in an array and not breaking anything in our code in the process.

Assigning Array Values

So far we have looked at various ways to pull data stored in an array using indexed elements. It's also possible to use a `for` loop to place multiple values into an array, very much like we did with our project source code. Take the following example:

```
int analogIn[6];
for (int i=0; i<6; i++) analogIn[i] = analogRead(i);
```

Here in the first, line we initialize an array called `analogIn[]` with a dimension of six elements, but we don't yet know what the values will be for each element. In the next line, we use a `for` loop to step through each of the six analog in pins, assigning a reading from each pin to each element in the array respectively. By placing these values into a single array, we can access those values in a multitude of ways later in our code.

Character Arrays

In the last few pages, each of our arrays has been used to store numerical data but it is also possible to store strings of text in an array, like we did earlier with the answers from our code in Project 8. A simple character array might look like the following:

```
char helloWorld[] = "Hello, world!";
```

This array is an example of C-style text strings of the `char` data type and contains the text `Hello, world!` The Arduino team has recently added a String object that offers more advanced functionality, but that's not really needed here. Essentially, the string of text is a collection of 14 elements as each letter of the string is stored as a separate element in the array. There are technically 14 elements even though there are only 13 characters, because all text strings need a final null character or 0 byte to tell any of the other functions where the string ends. Without this, functions like `Serial.print()` would continue to access parts of the memory that are used by other things. Fortunately, the Arduino compiler takes care of this for us adding the null character and determining the proper dimension of the array. If somehow we were to define the dimension of an array without considering this null character, the weird things will return and cause some unusual problems with our code.

Because each letter or character in a string of text is an individual element in a larger array, we could declare this array as follows:

```
char helloWorld[] = {'H', 'e', 'l', 'l', 'o', ',', ' ', 'w', 'o', 'r', 'l', 'd', '!'};
```

While functionally the same, I can tell you the second one was no fun to type. You might notice that individual characters declared in a character array use the single quotes as in `'H'` where as the text string is defined in double quotes as in `"Hello, world!"` instead. It is even possible to create an array of text strings as follows:

```
char* helloWorld[] = {"Hello,", "world!"};
```